

1 React `useRef` – Beginner to Intermediate Notes

2 Introduction

`useRef` is a React Hook that lets us:

- Access a DOM element directly.
- Store a value that does not cause the component to re-render.

Syntax:

```
const myRef = useRef(initialValue);
```

Example:

```
const inputRef = useRef(null);
```

Initially:

```
inputRef.current = null
```

After React renders the component and the ref is attached:

```
inputRef.current = <input>
```

`current` now contains the actual DOM element.

3 Purpose of `useRef`

`useRef` has two main purposes.

4 1. Access a DOM Element

Sometimes we need to directly interact with an HTML element.

Examples:

- Focus an input
- Play a video
- Pause a video
- Scroll to an element

- Measure element width or height

Example:

```
import { useRef } from "react";

function App() {
  const inputRef = useRef(null);

  function handleClick() {
    inputRef.current.focus();
  }

  return (
    <>
      <input ref={inputRef} />
      <button onClick={handleClick}>Focus</button>
    </>
  );
}
```

Explanation:

- `useRef()` creates a reference object.
 - `ref={inputRef}` connects the reference to the input element.
 - `inputRef.current` becomes the actual HTML input element.
 - Calling `focus()` moves the cursor into the input.
-

5 2. Store Values Without Re-rendering

Sometimes we need to remember a value, but we do not want React to re-render the component.

Example:

```
const timerRef = useRef(null);
```

Store timer ID:

```
timerRef.current = setInterval(() => {
  console.log("Running...");
}, 1000);
```

Stop timer:

```
clearInterval(timerRef.current);
```

The timer ID is stored inside `current`.

The user never sees this value, but the program needs it.

6 How useRef Works

Step 1

```
const inputRef = useRef(null);
```

React creates:

```
inputRef  
  
↓  
  
{  
  current: null  
}
```

Step 2

```
<input ref={inputRef} />
```

After rendering:

```
inputRef  
  
↓  
  
{  
  current: <input>  
}
```

Now React stores the actual input element inside `current`.

7 Example 1 – Auto Focus Input

```
import { useRef } from "react";  
  
function App() {  
  const inputRef = useRef(null);  
  
  function focusInput() {  
    inputRef.current.focus();  
  }  
}
```

```
    return (  
      <>  
        <input ref={inputRef} />  
        <button onClick={focusInput}>Focus</button>  
      </>  
    );  
  }  
}
```

What happens?

Button Click

↓

`focusInput()`

↓

`inputRef.current.focus()`

↓

Cursor moves into the input box.

8 Example 2 – Stopwatch

```
const timerRef = useRef(null);  
  
timerRef.current = setInterval(() => {  
  setSeconds(prev => prev + 1);  
}, 1000);  
  
clearInterval(timerRef.current);
```

Why `useRef`?

The timer ID must be remembered.

The timer ID is not displayed on the screen.

Therefore, `useRef` is the correct choice.

9 Example 3 – Video Player

```
const videoRef = useRef(null);

function playVideo() {
  videoRef.current.play();
}

function pauseVideo() {
  videoRef.current.pause();
}
```

JSX:

```
<video ref={videoRef}>
  <source src="sample.mp4" />
</video>
```

Flow:

Play Button

↓

videoRef.current

↓

Video Element

↓

play()

Pause:

Pause Button

↓

videoRef.current

↓

pause()

10 useState vs useRef

useState

Stores data displayed in the UI

Causes re-render

Uses setter function

Used for counters, forms, themes

useRef

Stores values not displayed in the UI

Does not cause re-render

Updates `.current` directly

Used for timers, DOM elements, previous values

Example:

useState

```
const [count, setCount] = useState(0);  
  
setCount(5);
```

React updates the UI.

useRef

```
const countRef = useRef(0);  
  
countRef.current = 5;
```

React does not update the UI.

11 Real-World Uses of useRef

- Auto-focus login input
- Search bar focus
- Stopwatch timer ID
- Play and pause videos
- Scroll to bottom in chat applications
- Scroll to a specific section
- Store previous value
- Store interval IDs
- Store timeout IDs
- Measure element size

12 When Should We Use useRef?

Use `useRef` when:

- You need direct access to a DOM element.
- You need to store a value without triggering a re-render.
- The value is only needed by the program, not displayed to the user.

Examples:

- Timer ID
 - Previous value
 - Video element
 - Audio element
 - Input element
 - Scroll position
-

13 When Should We Use `useState` Instead?

Use `useState` when:

- The value must appear in the UI.
- The UI should update whenever the value changes.

Examples:

- Counter
 - User name
 - Theme
 - Shopping cart
 - Todo list
 - Form inputs
-

14 Easy Analogy

15 `useState` = Notice Board

When the notice changes, everyone can see the update.

Notice Changed

↓

React Re-renders

↓

16 useRef = Personal Drawer

A value is stored inside a drawer.

People outside cannot see it.

Drawer

↓

Value Changes

↓

No UI Update

The application remembers the value, but React does not re-render.

17 Key Points

- `useRef` returns an object.
 - The object contains one property called `current`.
 - `current` stores the actual DOM element or any mutable value.
 - Updating `current` does not trigger a re-render.
 - `useRef` is mainly used for DOM access and storing values that should persist between renders.
-

18 Golden Interview Definition

`useRef` is a React Hook that allows us to access DOM elements directly and store mutable values that persist between renders without causing the component to re-render.